```
def lossfn(x,y):                        1
    loss = torch.Tensor([0.])
    loss.requires_grad=True
    for i in range(x.shape[0]):         2
        loss_ = -1 *  torch.log(x[i].flatten(start_dim=0)) @
    y[i].flatten(start_dim=0)           3
        loss = loss + loss_
    return loss
```

- ***1*** Loss between prediction distribution x and target distribution y
- ***2*** Loops through batch dimension
- ***3*** Flattens along action dimension to get a concatenated sequence of the distributions

The `lossfn` function takes a prediction distribution, `x`, of dimensions $B \times 3 \times 51$ and a target distribution, `y`, of the same dimensions, and then it flattens the distribution over the action dimension to get a $B \times 153$ matrix. Then it loops through each 1×153 row in the matrix and computes the cross entropy between the 1×153 prediction distribution and the 1×153 target distribution. Rather than explicitly summing over the product of `x` and `y`, we can combine these two operations and get the result in one shot by using the inner product operator, `@`.

We could choose to just compute the loss between the specific action-value distribution for the action that was taken, but we compute the loss for all three action-value distributions so that the Dist-DQN learns to keep the other two actions not taken unchanged; it only updates the action-value distribution that was taken.

7.6. Dist-DQN on simulated data

Let's test all the parts so far with a simulated target distribution to see if our Dist-DQN can successfully learn to match the target distribution. In listing 7.9 we take an initially uniform distribution, run it through our Dist-DQN, and update it using a synthetic vector of two reward observations.

Listing 7.9. Testing with simulated data